

Documentazione AnimeTracker

Traccia d'Esame — Progetto di un Sistema Informativo per la piattaforma [AnimeTracker](#)

Il seguente progetto riguarda l'analisi e la realizzazione di un sistema informativo per la piattaforma "AnimeTracker", un portale web pensato per la gestione e la condivisione di esperienze da parte di appassionati di anime.

Il progetto nasce con l'obiettivo di fornire uno strumento digitale moderno, funzionale e accessibile, che consenta agli utenti di documentare e organizzare la propria esperienza di visione, promuovendo al contempo la community degli appassionati.

L'interfaccia è semplice e intuitiva, costituita da un database relazionale (SQLite) che gestisce le informazioni relative a:

- Dati personali degli utenti;
- Generi degli anime (Azione, Fantasy, Sci-Fi, Thriller, Slice of Life, Storico);
- Studi di produzione (Wit Studio, Madhouse, ufotable, Bones, ecc.);
- Catalogo anime disponibili sulla piattaforma (con studio, anno e generi associati);
- Episodi degli anime (numero, titolo, durata);
- Liste personali degli utenti (anime seguiti, completati, da vedere);
- Recensioni pubbliche degli utenti sugli anime;
- Calcolo della media voti per ogni anime.

Il sistema distingue due categorie principali di utenti:

- **Utenti ospiti (non autenticati):** possono consultare il catalogo anime, i generi, gli studi e le recensioni, ma non possono interagire con la piattaforma;
 - **Utenti registrati (autenticati):** possono gestire la propria lista personale, assegnare voti, aggiungere commenti e scrivere recensioni pubbliche.
-

1. Analisi e progettazione concettuale: Modello E/R

Per il progetto AnimeTracker sono stati sviluppati tre modelli E/R via via piu raffinati, che documentano il percorso di progettazione concettuale dell'entita UTENTE fino alla soluzione logica adottata: **Modello Base**, **Modello Intermedio** e **Modello Finale** (pagine seguenti).

Generalizzazione totale e sovrapposta dell'entita UTENTE

UTENTE_OSPITE UTENTE_REGISTRATO

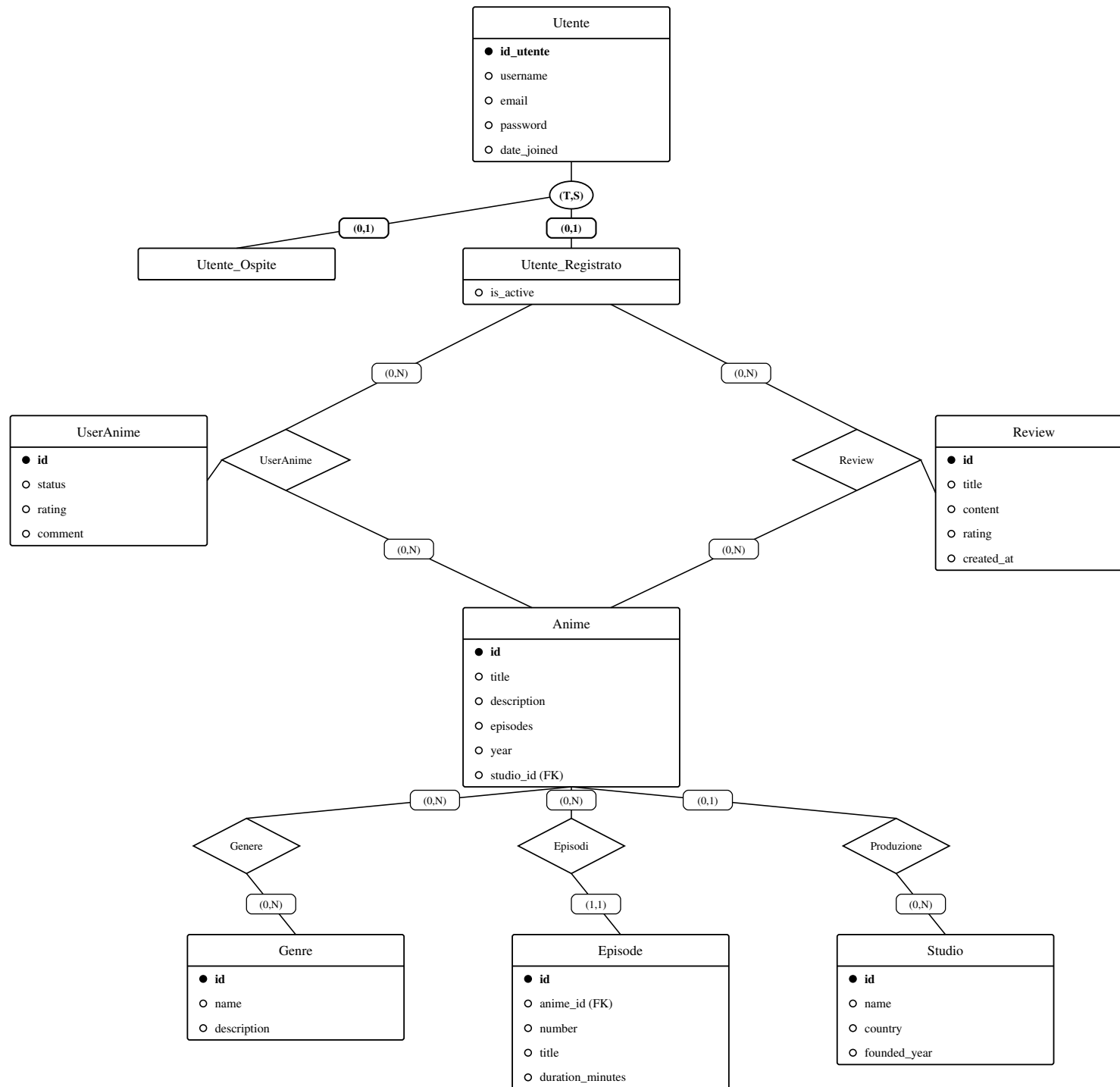
Il modello E/R prevede una generalizzazione totale e sovrapposta dell'entita UTENTE in due sottoclassi logiche:

- **UTENTE_OSPITE:** include tutti gli utenti non autenticati che possono consultare il catalogo anime, i generi, gli studi e le recensioni;
- **UTENTE_REGISTRATO:** rappresenta gli utenti autenticati abilitati alla gestione della lista personale, all'assegnazione di voti, alla scrittura di recensioni pubbliche.

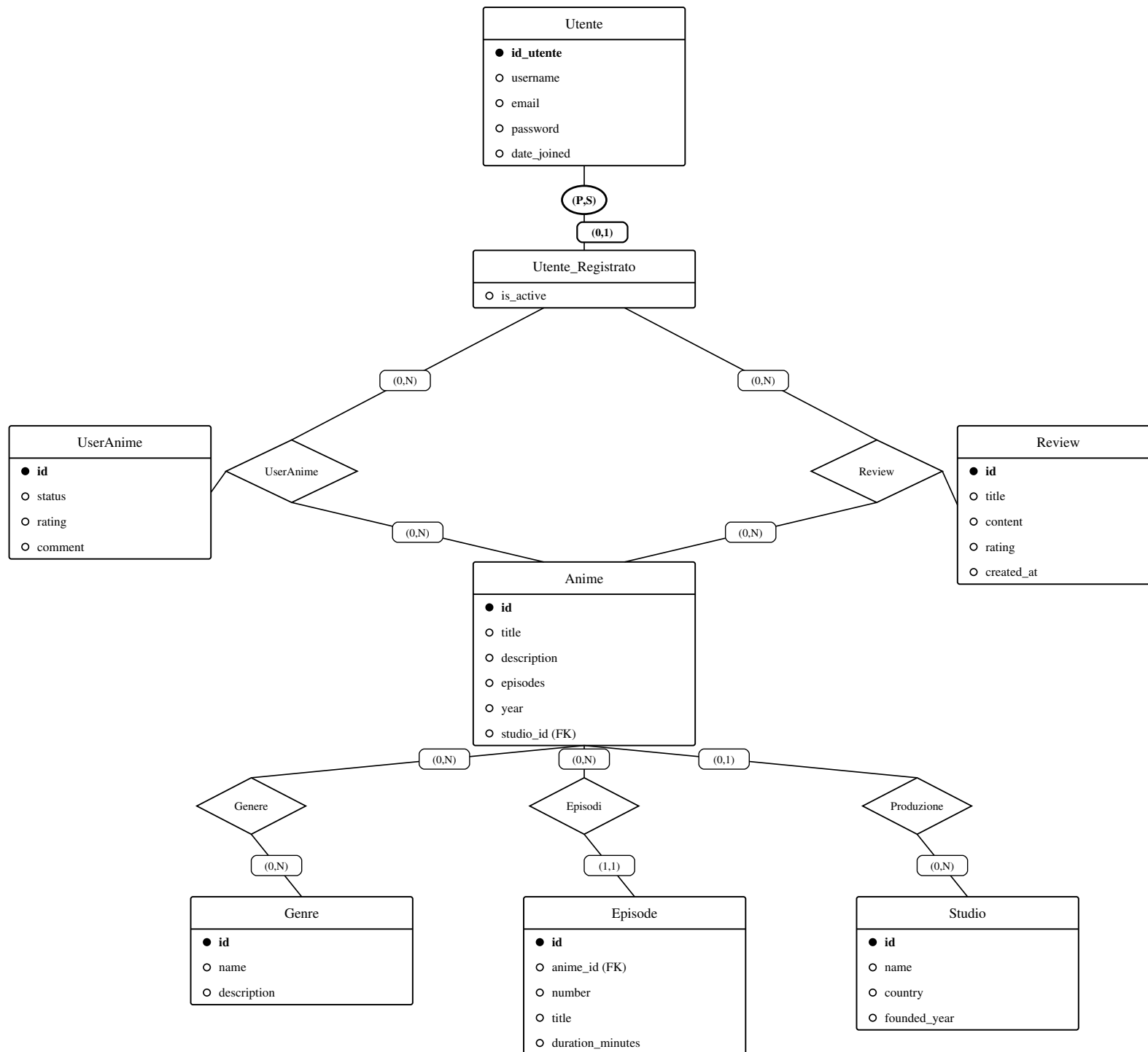
Tipo di generalizzazione:

- **Totale:** ogni istanza dell'entita UTENTE rientra obbligatoriamente in almeno una delle due sottoclassi.

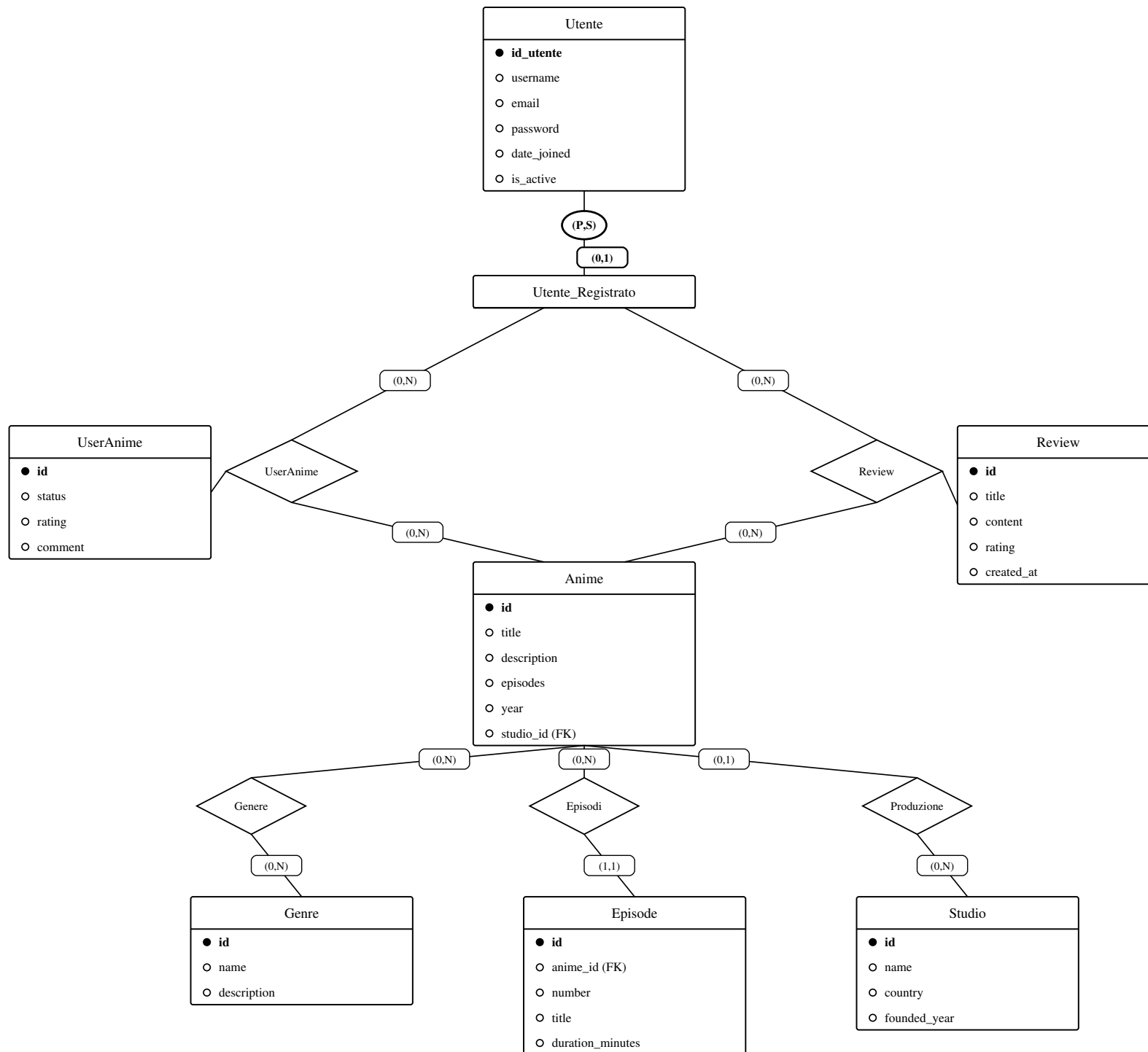
Base



Intermedio



Finale



- **Sovrapposta:** un utente registrato appartiene contemporaneamente a entrambe le categorie, in quanto può consultare i contenuti e gestire la propria lista personale.

Strategie di Risoluzione della Generalizzazione

1. Strategia "Tutto nel padre"

Si mantiene un'unica entità UTENTE che include tutti gli attributi necessari per distinguere i ruoli (in questo caso, tramite il flag di autenticazione Django).

```
UTENTE(id, username UNIQUE, email UNIQUE, password, date_joined, is_active)
```

- Nessuna tabella separata per UTENTE_OSPITE e UTENTE_REGISTRATO.
- I privilegi sono gestiti tramite il sistema di autenticazione Django.
- Le funzionalità disponibili sono controllate tramite decorator `@login_required`.

Vantaggi:

Schema semplificato, nessuna ridondanza.

Facilita la gestione di utenti che si registrano e cambiano stato nel tempo.

Django gestisce nativamente questo pattern tramite il modello `auth.User`.

Svantaggi:

La distinzione formale tra i ruoli non è visibile nel modello concettuale se non attraverso la logica applicativa.

2. Strategia "Tutto nelle figlie"

Questa soluzione prevede la creazione di due entità separate UTENTE_OSPITE e UTENTE_REGISTRATO, collegate all'entità padre UTENTE tramite una relazione 1:1.

```
UTENTE(id, username, email, password, date_joined)
```

```
UTENTE_OSPITE(id_utente) → FK verso UTENTE
```

```
UTENTE_REGISTRATO(id_utente, is_active) → FK verso UTENTE
```

Vantaggi:

La distinzione tra ruoli è esplicita nel modello e nella base di dati.

Svantaggi:

Non adatta in assenza di attributi distintivi tra le due categorie.

Introduce complessità non necessaria per un sistema semplice.

Strategia adottata

Nel progetto AnimeTracker, è stata adottata la strategia **"Tutto nel padre"**. Questo perché:

- Non esistono attributi distintivi tra UTENTE_OSPITE e UTENTE_REGISTRATO;
- La differenza tra i ruoli è gestita dinamicamente dal sistema di autenticazione integrato di Django;
- Gli utenti possono registrarsi e ottenere nuovi privilegi in modo immediato;
- Il sistema gestisce i permessi tramite il decorator `@login_required` a livello applicativo in modo flessibile.

2. Progettazione Logica

La progettazione logica del database è stata sviluppata a partire dal modello concettuale E/R, adottando una generalizzazione totale e sovrapposta per l'entità Utente. Il sistema è composto da **6 entità principali** collegate tramite relazioni FK e M2M.

`Utente:(id, username UNIQUE, email UNIQUE, password, date_joined)`

→ Rappresenta gli utenti registrati sulla piattaforma. Gestito da `django.contrib.auth.models.User`.

`Genre:(id, name UNIQUE, description)`

→ Generi degli anime. Collegato ad Anime tramite relazione M:N.

`Studio:(id, name UNIQUE, country, founded_year)`

→ Studio di produzione. Collegato ad Anime tramite FK.

`Anime:(id, title, description, episodes, year, studio_id:Studio)`

→ Ogni anime è un elemento del catalogo. Collegato allo Studio tramite FK e ai Genre tramite M2M.

`Episode:(id, anime_id:Anime, number, title, duration_minutes)`

→ Episodi di un anime. Vincolo di unicità: (anime_id, number).

`UserAnime:(id, user_id:Utente, anime_id:Anime, status, rating, comment)`

→ Entità associativa che realizza la relazione M:N tra Utente e Anime.

`Review:(id, user_id:Utente, anime_id:Anime, title, content, rating, created_at)`

→ Recensione pubblica. Vincolo: (user_id, anime_id) univoca.

UTENTE (auth_user — Django built-in)

Nome campo	Descrizione	Tipo dati	Lunghezza	Vincoli
<code>id</code>	Identificatore univoco utente	INT	—	PK, AUTO_INCREMENT
<code>username</code>	Nome utente	VARCHAR	150	NOT NULL, UNIQUE
<code>email</code>	Indirizzo email	VARCHAR	254	NOT NULL, UNIQUE
<code>password</code>	Password cifrata (PBKDF2)	VARCHAR	128	NOT NULL
<code>date_joined</code>	Data di registrazione	DATETIME	—	NOT NULL, DEFAULT NOW()
<code>is_active</code>	Utente attivo	BOOLEAN	—	NOT NULL, DEFAULT TRUE

Vincoli:

`username` e `email` devono essere univoci nel sistema.

La password viene automaticamente cifrata da Django con algoritmo PBKDF2+SHA256.

GENRE

Nome campo	Descrizione	Tipo dati	Lunghezza	Vincoli
id	Identificatore univoco genere	INT	—	PK, AUTO_INCREMENT
name	Nome del genere	VARCHAR	100	NOT NULL, UNIQUE
description	Descrizione del genere	TEXT	—	NULLABLE

Vincoli:

name deve essere univoco nel sistema (es. non possono esistere due generi "Azione").

description è opzionale (NULLABLE).

STUDIO

Nome campo	Descrizione	Tipo dati	Lunghezza	Vincoli
id	Identificatore univoco studio	INT	—	PK, AUTO_INCREMENT
name	Nome dello studio di produzione	VARCHAR	200	NOT NULL, UNIQUE
country	Paese di origine	VARCHAR	100	NOT NULL, DEFAULT 'Giappone'
founded_year	Anno di fondazione	INT	—	NULLABLE

Vincoli:

name deve essere univoco.

founded_year è opzionale.

ANIME

Nome campo	Descrizione	Tipo dati	Lunghezza	Vincoli
id	Identificatore univoco anime	INT	—	PK, AUTO_INCREMENT
title	Titolo dell'anime	VARCHAR	200	NOT NULL
description	Descrizione/trama	TEXT	—	NULLABLE
episodes	Numero episodi totali	INT	—	NOT NULL, CHECK (episodes > 0)
year	Anno di prima messa in onda	INT	—	NULLABLE
studio_id	FK → Studio di produzione	INT	—	NULLABLE, FK → Studio(id) ON DELETE SET NULL

Vincoli:

`title` è obbligatorio (NOT NULL).
`description` , `year` e `studio_id` sono opzionali (NULLABLE).
`episodes` deve essere un intero positivo.
Se lo studio viene eliminato, `studio_id` viene impostato a NULL (SET NULL).

EPIISODE

Nome campo	Descrizione	Tipo dati	Lunghezza	Vincoli
<code>id</code>	Identificatore univoco episodio	INT	—	PK, AUTO_INCREMENT
<code>anime_id</code>	FK → Anime di appartenenza	INT	—	NOT NULL, FK → Anime(id) ON DELETE CASCADE
<code>number</code>	Numero episodio nella serie	INT	—	NOT NULL, CHECK (number > 0)
<code>title</code>	Titolo dell'episodio	VARCHAR	300	NULLABLE
<code>duration_minutes</code>	Durata in minuti	INT	—	NULLABLE, CHECK (duration_minutes > 0)
<code>(coppia univoca)</code>	Unicità episodio per anime	—	—	UNIQUE(anime_id, number)

Vincoli:

La coppia `(anime_id, number)` deve essere univoca: non possono esistere due episodi con lo stesso numero per lo stesso anime.
Se l'anime viene eliminato, tutti i suoi episodi vengono cancellati automaticamente (CASCADE).

USER_ANIME (entità associativa)

Nome campo	Descrizione	Tipo dati	Lunghezza	Vincoli
<code>id</code>	Identificatore univoco	INT	—	PK, AUTO_INCREMENT
<code>user_id</code>	FK utente proprietario	INT	—	NOT NULL, FK → Utente(id) ON DELETE CASCADE
<code>anime_id</code>	FK anime riferito	INT	—	NOT NULL, FK → Anime(id) ON DELETE CASCADE
<code>status</code>	Stato di visione	VARCHAR	20	NOT NULL, CHECK IN ('watching','completed','planned')
<code>rating</code>	Voto da 1 a 10	INT	—	NULLABLE, CHECK (rating BETWEEN 1 AND 10)
<code>comment</code>	Commento personale	TEXT	—	NULLABLE
<code>(coppia univoca)</code>	Unicità utente/anime	—	—	UNIQUE(user_id, anime_id)

Vincoli:

`status` deve appartenere all'insieme {'watching', 'completed', 'planned'}.

`rating` e `comment` sono opzionali (NULLABLE).

`rating` , se specificato, deve essere compreso tra 1 e 10.

La coppia `(user_id, anime_id)` deve essere univoca: un utente può avere al massimo una voce per ogni anime.

REVIEW (recensione pubblica)

Nome campo	Descrizione	Tipo dati	Lunghezza	Vincoli
<code>id</code>	Identificatore univoco	INT	—	PK, AUTO_INCREMENT
<code>user_id</code>	FK utente autore	INT	—	NOT NULL, FK → Utente(id) ON DELETE CASCADE
<code>anime_id</code>	FK anime recensito	INT	—	NOT NULL, FK → Anime(id) ON DELETE CASCADE
<code>title</code>	Titolo della recensione	VARCHAR	200	NOT NULL
<code>content</code>	Testo completo della recensione	TEXT	—	NOT NULL
<code>rating</code>	Voto della recensione (1–10)	INT	—	NOT NULL, CHECK (rating BETWEEN 1 AND 10)
<code>created_at</code>	Data/ora di pubblicazione	DATETIME	—	NOT NULL, DEFAULT NOW() (auto)
(coppia univoca)	Unicità utente/anime	—	—	UNIQUE(user_id, anime_id)

Vincoli:

`title`, `content` e `rating` sono obbligatori.

`rating` deve essere compreso tra 1 e 10.

La coppia (`user_id`, `anime_id`) deve essere univoca: un utente può scrivere al massimo una recensione per ogni anime.

`created_at` viene impostato automaticamente al momento della creazione.

Vincoli interrelazionali

Unicità voce per utente/anime

Un utente può inserire al massimo una voce nella propria lista per ogni anime.

→ Coinvolge: UserAnime.

→ Implementato con `UNIQUE(user_id, anime_id)` + `get_or_create()` a livello applicativo.

Unicità recensione per utente/anime

Un utente può scrivere al massimo una recensione per ogni anime.

→ Coinvolge: Review.

→ Implementato con `UNIQUE(user_id, anime_id)`.

Accesso alla lista personale riservato agli utenti autenticati

Solo gli utenti autenticati possono aggiungere, modificare o rimuovere anime dalla lista e scrivere recensioni.

→ Gestito tramite decorator `@login_required` su tutte le view di gestione.

Cancellazione a cascata delle voci lista

Se un utente viene eliminato, tutte le sue voci UserAnime e Review vengono cancellate automaticamente.

→ Implementato tramite `on_delete=CASCADE` sulla FK `UserAnime.user` e `Review.user`.

Cancellazione a cascata degli anime

Se un anime viene eliminato dal catalogo, tutte le voci UserAnime, Review ed Episode associate vengono cancellate automaticamente.

→ Implementato tramite `on_delete=CASCADE` sulle FK verso Anime.

3. Implementazione del sistema informativo

L'implementazione del sistema informativo AnimeTracker è stata sviluppata utilizzando il framework Django, selezionato per la sua scalabilità, sicurezza e semplicità di manutenzione. Il database utilizzato è SQLite, soluzione integrata in Django adatta per applicazioni di dimensioni contenute.

Il sistema organizza i dati secondo il modello logico definito con 6 entità, garantendo integrità e coerenza, e risponde alle esigenze degli appassionati di anime assicurando correttezza nelle operazioni di gestione delle liste personali e delle recensioni.

◆ Funzionalità principali del sistema AnimeTracker

1. Registrazione e accesso sicuro

AnimeTracker offre un sistema di autenticazione basato sul sistema integrato di Django. Durante la registrazione, l'utente inserisce username, email e password. Le password vengono cifrate con algoritmo PBKDF2 + SHA256, standard Django. Il login è gestito tramite `django.contrib.auth.authenticate()` e `login()`. Il logout invalida la sessione tramite `django.contrib.auth.logout()`.

2. Catalogo anime e ricerca per genere

Tutti gli utenti (ospiti e registrati) possono consultare il catalogo completo degli anime. La ricerca per titolo è realizzata tramite `filter(title__icontains=query)`. Il filtro per genere filtra tramite `filter(genres__id=genre_id)`. Per ogni anime viene mostrata la media dei voti, calcolata dinamicamente tramite il metodo `average_rating()` sul modello.

3. Scheda dettaglio anime

Ogni anime ha una pagina dedicata con: studio di produzione, anno, generi, elenco episodi e tutte le recensioni degli utenti. L'utente autenticato può aggiungere la propria voce in lista o scrivere una recensione direttamente dalla scheda.

4. Lista personale e gestione stati

Ogni utente registrato ha una lista personale dove può inserire, modificare e visualizzare i propri anime. Il form consente di impostare lo stato di visione (In visione / Completato / Da vedere), il voto (1-10) e un commento. La pagina lista personale supporta il filtro per stato tramite parametro GET.

5. Recensioni pubbliche

Gli utenti autenticati possono scrivere una recensione pubblica per ogni anime (titolo, testo, voto 1-10). Le recensioni sono visibili a tutti nella scheda dettaglio. Un utente può modificare o eliminare la propria recensione. Il vincolo `unique_together(user, anime)` garantisce al massimo una recensione per utente per anime.

6. Pannello amministrazione

L'interfaccia admin di Django (`/admin/`) consente ai superuser di gestire il catalogo anime, gli utenti e tutte le entità: Genre, Studio, Anime, Episode, UserAnime, Review.

 **AnimeTracker**

 Lista Anime

 Generi

 Studi

Login

Registrati

**Bentornato**

Accedi al tuo account AnimeTracker

Accedi

- La schermata di login consente l'accesso con username e password.
- In caso di credenziali errate, viene mostrato un messaggio di errore.
- Il pulsante "Registrati" reindirizza alla pagina di signup con form completo.

CATALOGO ANIME

 **AnimeTracker**

 Lista Anime

 Generi

 Studi

 La Mia Lista

 mario Esci

 **Lista Anime**

12 anime disponibili

Cerca titolo...

Tutti

Azione

Fantasy

Sci-Fi

Thriller

Storico

TITOLO	STUDIO	ANNO	EP.	GENERI	MEDIA VOTI	AZIONE
Attack on Titan	Wit Studio	2013	▶ 87 ep.	Azione Fantasy	★ 9.2/10	 Modifica
Death Note	Madhouse	2006	▶ 37 ep.	Thriller	★ 8.8/10	+ Aggiungi
Steins;Gate	J.C.Staff	2011	▶ 24 ep.	Sci-Fi Thriller	★ 9.5/10	+ Aggiungi

- La pagina principale mostra tutti gli anime con titolo, studio, anno, episodi, generi, media voti e azione disponibile.
- I filtri rapidi per genere permettono di filtrare il catalogo.
- La barra di ricerca filtra gli anime per titolo in tempo reale (query GET).
- Il pulsante "Aggiungi" porta al form di inserimento nella lista personale. Se l'anime è già in lista, il pulsante diventa "Modifica" (verde).

SCHEDA DETTAGLIO ANIME

 **AnimeTracker**

 Lista Anime

 Generi

 Studi

 La Mia Lista

 mario

Esci

Attack on Titan

Wit Studio · 2013 · 87 episodi

 **9.2/10**

 La mia lista

Descrizione

L'umanità sopravvive dietro enormi mura per proteggersi dai Titani, giganteschi esseri che divorano gli esseri umani senza apparente motivo.

Scheda

Studio	Wit Studio
Anno	2013
Episodi	87
Generi	Azione Fantasy

Episodi (3 mostrati)

1	Per te, duemila anni dopo	24 min
2	Quel giorno	24 min
3	Notte del giuramento	24 min

Recensioni (1)

+ Scrivi recensione

Capolavoro assoluto  **10/10**

mario · 23/05/2026 

Una storia straordinaria che ti tiene incollato allo schermo. La narrativa è incredibilmente ben costruita.

- La scheda dettaglio mostra tutte le informazioni dell'anime: studio, anno, episodi, generi.
- Vengono elencati tutti gli episodi con numero, titolo e durata.
- Le recensioni pubbliche degli utenti sono visibili con voto, testo e data.
- L'utente autenticato può scrivere, modificare o eliminare la propria recensione.

LA MIA LISTA

 **AnimeTracker**

 Lista Anime

 Generi

 Studi

 La Mia Lista

 mario

Esci

 **La Mia Lista**

3 anime nella tua collezione

Tutti

 In visione

 Completato

 Da vedere

TITOLO	STATO	VOTO	AZIONI
Steins;Gate	 Completato	 10/10	 
One Punch Man	 In visione	—	 
Vinland Saga	 Da vedere	—	 

- La lista personale mostra tutti gli anime aggiunti dall'utente con stato, voto e azioni disponibili.
- I filtri rapidi per stato (In visione / Completato / Da vedere) permettono di filtrare la visualizzazione.
- Ogni riga ha pulsanti per modificare o rimuovere l'anime dalla lista.

Gestione delle sessioni e controllo accessi

Nel progetto AnimeTracker, la gestione dello stato di autenticazione è basata sul sistema di sessioni e autenticazione integrato di Django (`django.contrib.auth`). Quando un utente effettua il login, Django aggiorna automaticamente la sessione HTTP con le informazioni dell'utente autenticato. Le view protette utilizzano il decorator `@login_required` :

```

from django.contrib.auth.decorators import login_required

@login_required
def my_list(request):
    entries = UserAnime.objects.filter(user=request.user).select_related('anime')
    return render(request, 'anime/my_list.html', {'entries': entries})

@login_required
def add_review(request, anime_id):
    anime = get_object_or_404(Anime, pk=anime_id)
    existing = Review.objects.filter(user=request.user, anime=anime).first()
    # un solo record Review per (user, anime) - unique_together garantisce l'unicità
    if request.method == 'POST':
        form = ReviewForm(request.POST, instance=existing)
        if form.is_valid():
            review = form.save(commit=False)
            review.user = request.user
            review.anime = anime
            review.save()

```

Se un utente non autenticato tenta di accedere a una view protetta, viene automaticamente reindirizzato alla pagina di login. Il logout invalida completamente la sessione tramite:

```

from django.contrib.auth import logout

def logout_view(request):
    logout(request) # invalida la sessione e rimuove i dati di autenticazione
    return redirect('anime_list')

```

Simulazione di SQL Injection

Supponiamo di avere un codice che gestisce il login dell'utente costruendo manualmente la query SQL in questo modo:

```

from django.db import connection

def login_vulnerabile(request):
    if request.method == "POST":
        username = request.POST["username"]
        password = request.POST["password"]
        with connection.cursor() as cursor:
            # Costruzione pericolosa: input inserito direttamente nella query
            query = f"SELECT * FROM auth_user WHERE username = '{username}' AND password = '{password}'"
            cursor.execute(query)
            user = cursor.fetchone()
            if user:
                pass # login riuscito
            else:
                pass # login fallito

```

Come avviene l'attacco

- Un utente malintenzionato inserisce come username: `admin' OR '1'='1'`
- E come password: `qualunque_cosa`

La query risultante diventa:

```

SELECT * FROM auth_user WHERE username = 'admin' OR '1'='1' AND password = 'qualunque_cosa'

```

Per la logica SQL, `'1'='1'` è sempre vera, quindi la query ritorna tutti gli utenti o almeno uno, permettendo l'accesso senza conoscere la password corretta.

Vulnerabilità introdotte

- **Concatenazione diretta:** l'input dell'utente viene inserito direttamente nella stringa SQL senza sanitizzazione.
- **Nessuna separazione dati/codice:** i dati (input utente) e il codice SQL sono mescolati, così un input malevolo diventa parte del codice eseguibile.
- **Possibilità di manipolazione della query:** l'attaccante può modificare la logica della query a suo favore.

Test dell'iniezione con SQLmap

Strumenti come SQLmap automatizzano gli attacchi SQL Injection. Se il form di login vulnerabile invia una richiesta POST come questa:

```

POST /login/ HTTP/1.1
Host: localhost:8000
Content-Type: application/x-www-form-urlencoded
username=admin&password=1234

```

Salvando questa richiesta in un file (`req.txt`), SQLmap può essere lanciato così:

```
sqlmap -r req.txt --risk=3 --level=5 --dump
```

- `-r req.txt` : carica una richiesta HTTP completa da file.
- `--risk=3` : imposta il grado di rischio dei test (3=alto).
- `--level=5` : definisce il livello di profondità; 5 controlla tutti i parametri possibili.
- `--dump` : se trova una vulnerabilità, estrae tutti i dati dal database.

Risultato: se la query è vulnerabile, l'attaccante ottiene l'accesso o scarica l'intero database.

Soluzioni SQL Injection

Uso dell'ORM Django

Django fornisce un Object Relational Mapper (ORM) che permette di interagire con il database usando metodi Python, senza scrivere query SQL manuali.

Esempio sicuro — autenticazione con ORM:

```
from django.contrib.auth import authenticate

def login_sicuro(request):
    if request.method == "POST":
        username = request.POST["username"]
        password = request.POST["password"]
        # L'ORM gestisce automaticamente escaping e sanitizzazione
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user) # login riuscito
        else:
            pass # credenziali errate
```

È sicuro perché:

- L'ORM gestisce automaticamente l'escaping e la sanitizzazione degli input.
- I valori non vengono mai interpretati come codice SQL, ma solo come dati da cercare.
- Anche input malevoli vengono trattati come semplici stringhe, senza influenzare la query.

Uso di query parametrizzate

Se si deve scrivere query SQL manualmente, è fondamentale usare query parametrizzate per separare il codice dai dati:

```
from django.db import connection

def login_sicuro_raw(request):
    if request.method == "POST":
        username = request.POST["username"]
        password = request.POST["password"]
        with connection.cursor() as cursor:
            # I parametri sono passati separatamente come lista
            query = "SELECT * FROM auth_user WHERE username = %s AND password = %s"
            cursor.execute(query, [username, password])
            user = cursor.fetchone()
```

È sicuro perché i parametri (`username` e `password`) sono passati separatamente e l'adapter del database li gestisce senza esporre la query a manipolazione. Nessun input utente può alterare la struttura della query SQL.